



MASTER 1 MIAGE

TP-SIR

Notice d'utilisation du site

EventHub

Réalisé par :

MENDES A D Mamy
NOUBI-SI Ines

Encadré par :

Le Houedec Fabien

Architecture :

Angular 18 + JAX-RS + MySQL

Année Universitaire 2025-2026

TABLE DES MATIÈRES

1. Introduction	3
2. Présentation de l'application	3
3. Architecture technique	4
3.1 Vue d'ensemble (Angular + JAX-RS + MySQL)	4
3.2 Modèle de données — Entités JPA	5
3.3 Relations ManyToMany Événement ↔ Artiste	6
3.4 Authentification JWT	7
3.5 Gestion des rôles (RBAC)	8
3.6 Upload et affichage des images	9
3.7 Architecture Angular 18 — Structure, Signals, Proxy, Guards	10
4. Démarrage de l'application	12
4.1 Prérequis	10
4.2 Lancement du backend (Docker)	10
4.3 Lancement du frontend (Angular)	11
5. Comptes utilisateurs disponibles	12
6. Fonctionnalités — Guide utilisateur	12
6.1 Page d'accueil	12
6.2 Connexion et inscription	13
6.3 Réservation de tickets (3 étapes)	14
6.4 QR Code et génération PDF	15
6.5 Contrôle des tickets	16
6.6 Gestion des artistes	17
6.7 Espace Organisateur	18
6.8 Panneau Administrateur + Statistiques	19
7. API Backend — Endpoints complets	20
7.1 Auth (login / register)	20
7.2 Événements	20
7.3 Artistes	21
7.4 Tickets	21
7.5 Utilisateurs & Organismes	22
8. CRUD complet — Détail des opérations	23
9. Statistiques et KPIs	24
10. Problèmes rencontrés et solutions	25
11. Conclusion	27

1. INTRODUCTION

Ce document constitue le guide technique et fonctionnel complet de l'application EventHub, développée dans le cadre de notre projet.

Il couvre l'ensemble de l'architecture logicielle, toutes les fonctionnalités implémentées, les choix techniques réalisés, les problèmes rencontrés et leurs solutions, ainsi qu'un guide pas à pas pour l'utilisation de chaque page de l'application.

Stack technique : Angular 18 (frontend) • JAX-RS / Jakarta EE (backend REST) • JPA / Hibernate (persistance) • MySQL / HSQLDB (base de données) • Docker (déploiement) • JWT (authentification)

2. PRÉSENTATION DE L'APPLICATION

EventHub est une plateforme web complète de gestion et de réservation d'événements musicaux. Elle s'adresse à trois types d'acteurs distincts aux besoins complémentaires.

Acteur	Rôle	Fonctionnalités clés
 Utilisateur	Spectateur / Acheteur	Parcourir, réserver, télécharger PDF, QR code
 Organisateur	Créateur d'événements	CRUD événements, ajout artistes, statistiques
 Administrateur	Gestionnaire global	Dashboard KPI, gestion users, accès total

Palette de couleurs

Code HEX	Couleur	Usage
#C82909	Rouge vif	Boutons primaires, accents, titres de section
#FCC6BB	Rose clair	Badges, fonds secondaires, encadrés info
#E7DED9	Beige chaud	Fonds de page, arrière-plans neutres
#2D211C	Brun foncé	Textes, titres principaux

3. ARCHITECTURE TECHNIQUE

3.1 Vue d'ensemble — Angular + JAX-RS + MySQL

L'application repose sur une architecture trois-tiers classique, avec une séparation nette entre présentation, logique métier et persistance.

```
FRONTEND → Angular 18 (TypeScript, Services, Guards, Interceptors)    ↑ HTTP REST JSON
(port 4200 → 8081) BACKEND → JAX-RS / Jakarta EE (Resources, Services, DAOs, JPA)    ↑
JDBC / JPA / Hibernate BASE DE DONNÉES → MySQL (prod) / HSQLDB (dev/test via Docker)
```

Packages backend (fr.istic.taa.jaxrs)

Package	Rôle	Classes principales
domain	Entités JPA (modèle)	Evenement, Artiste, Ticket, Utilisateur, Organisateur, Admin
dto	Objets de transfert	EvenementDto, ArtisteDto, TicketDto, LoginRequest/Response
dao.generic	Couche accès données	AbstractJpaDao, EvenementDao, ArtisteDao, TicketDao...
service	Logique métier	EvenementService, ArtisteService, TicketService...
rest	Ressources JAX-RS (endpoints)	EvenementResource, ArtisteResource, TicketResource, AuthResource
security	JWT + Filtres	JwtUtil, JwtFilter
configuration	CORS + Jackson	CorsFilter, CorsOptionsRequestFilter, JacksonConfig

3.2 Modèle de données — Entités JPA

Le modèle objet est organisé autour d'une hiérarchie d'héritage pour les personnes (table unique SINGLE_TABLE) et de relations explicites entre les entités métier.

Entité Personne (classe mère)

```
@Entity @Inheritance(strategy = InheritanceType.SINGLE_TABLE)
public abstract class Personne { Long idPersonne; String nom; String prenom;
String email; String password; String role; }
```

- Utilisateur extends Personne — login, historique tickets
- Organisateur extends Personne — nomStructure, numeroSiret, liste d'événements
- Admin extends Personne — dateNomination, actif

Entité Evenement

Champ	Type JPA	Description
idEvenement	@Id @GeneratedValue	Clé primaire auto-incrémentée
nom	String	Titre de l'événement
date	LocalDate	Date de l'événement
heure	String	Heure (ex: 20h30)
lieu	String	Lieu/salle
genre / categorie	String	Style musical et catégorie UI
capacite	Long	Nombre maximum de places
nbTicketsVendus	Long	Compteur de tickets vendus
prix	Double	Prix unitaire du billet
popularite	Float	Score de popularité (0–100)
description	String (TEXT)	Description longue
imageUrl	String	URL de l'image de couverture
organisateur	@ManyToOne	FK vers Organisateur
artistes	@ManyToMany	Table de jointure evenement_artiste
tickets	@OneToMany	Liste des tickets émis

Entité Artiste

Champ	Type	Description
idArtiste	Long (@Id)	Clé primaire
nom / prenom	String	Identité de l'artiste
styleArtistique	String	Genre musical (Jazz, Rock, Électro...)
nationalite	String	Pays d'origine
dateNaissance	LocalDate	Date de naissance
description	String (TEXT)	Biographie courte
popularite	Integer	Score de popularité
siteWeb	String	URL site officiel
imageUrl	String	Photo/visuel de l'artiste
evenements	@ManyToMany (mappedBy)	Événements associés (@JsonIgnore côté artiste)

Entité Ticket

Champ	Type	Description
idTicket	Long (@Id)	Clé primaire
numeroPlace	String	Numéro unique du billet
statut	StatutTicketEnum	VALIDE, UTILISE, ANNULE, REMBOURSE
prixUnitaire	Double	Prix payé
dateAchat	LocalDateTime	Horodatage d'achat
dateAnnulation	LocalDateTime	Horodatage d'annulation (nullable)
dateRemboursement	LocalDateTime	Horodatage de remboursement (nullable)
evenement	@ManyToOne	Référence à l'événement
utilisateur	@ManyToOne	Référence à l'utilisateur acheteur

3.3 Relation ManyToMany Événement ↔ Artiste

La relation entre événements et artistes est bidirectionnelle ManyToMany. Elle est matérialisée par une table de jointure intermédiaire `evenement_artiste` en base de données.

Côté Evenement (propriétaire de la relation)

```
@ManyToMany
@JoinTable(
    name = "evenement_artiste",
    joinColumns = @JoinColumn(name = "evenement_id"),
    inverseJoinColumns = @JoinColumn(name = "artiste_id")
)
private List<Artiste> artistes = new ArrayList<>();
```

Côté Artiste (inverse, lecture seule)

```
@JsonIgnore // évite la récursion infinie lors de la sérialisation JSON
@ManyToMany(mappedBy = "artistes")
private List<Evenement> evenements = new ArrayList<>();
```

⚠ Sans `@JsonIgnore` côté Artiste, la sérialisation JSON provoque une récursion infinie (`StackOverflowError`). C'est l'un des problèmes les plus fréquents avec les relations bidirectionnelles JPA.

Table de jointure SQL générée

```
CREATE TABLE evenement_artiste (
    evenement_id BIGINT NOT NULL,
    artiste_id BIGINT NOT NULL,
    PRIMARY KEY (evenement_id, artiste_id),
    FOREIGN KEY (evenement_id) REFERENCES evenement(id_evenement),
    FOREIGN KEY (artiste_id) REFERENCES artiste(id_artiste)
);
```

Ajout/suppression d'un artiste à un événement (API)

Méthode	URL	Description
POST	/evenements/{id}/artistes/{artisteld}	Associer un artiste à un événement
DELETE	/evenements/{id}/artistes/{artisteld}	Retirer un artiste d'un événement
GET	/evenements/with-artistes	Récupérer événements avec leurs artistes
GET	/artistes/{id}/evenements	Événements d'un artiste donné

3.4 Authentification JWT

L'authentification repose sur JSON Web Tokens (JWT). Après un login réussi, le serveur retourne un token signé que le client Angular inclut dans chaque requête protégée via le header Authorization.

Flux d'authentification

1. L'utilisateur envoie POST /auth/login avec { email, password }
2. Le serveur vérifie les credentials en base (Utilisateur ou Organisateur ou Admin)
3. Si valides → JwtUtil.generateToken(email, role) génère un token HS256 signé
4. Le token est retourné dans LoginResponse { token, user }
5. Angular stocke le token dans localStorage et l'injecte via un HttpInterceptor
6. Chaque requête protégée transporte le header Authorization: Bearer <token>
7. JwtFilter (ContainerRequestFilter) valide le token avant chaque appel API

Génération du token — JwtUtil.java

```
public static String generateToken(String email, String role) {
    return Jwts.builder()
        .setSubject(email)
        .claim("role", role)           // rôle encodé dans le payload
        .setIssuedAt(new Date())
        .setExpiration(new Date(now + 86_400_000)) // 24h
        .signWith(SignatureAlgorithm.HS256, SECRET_KEY.getBytes())
        .compact();
}
```

Validation du token — JwtFilter.java

```
@Provider @Priority(Priorities.AUTHENTICATION)
public class JwtFilter implements ContainerRequestFilter {
    public void filter(ContainerRequestContext ctx) {
        String path = ctx.getUriInfo().getPath();
        // Routes publiques : auth/**, evenements/**, artistes/**
        if (isPublicRoute(path)) return;
        String token = extractBearerToken(ctx);
        if (token == null) { ctx.abortWith(401); return; }
        JwtUtil.validateToken(token); // lève exception si invalide
    }
}
```

Routes publiques vs protégées

Route	Accès	JWT requis
GET /evenements/**	Public	Non
GET /artistes/**	Public	Non
POST /auth/login	Public	Non
POST /auth/register	Public	Non
POST /tickets	Utilisateur authentifié	Oui
PUT /tickets/{id}	Utilisateur authentifié	Oui
POST /evenements	Organisateur / Admin	Oui
DELETE /evenements/{id}	Organisateur / Admin	Oui
GET /utilisateurs	Admin uniquement	Oui

3.5 Gestion des rôles (RBAC)

Trois rôles distincts sont définis dans l'application. Le contrôle d'accès est géré à deux niveaux : côté Angular (guards de routes) et côté backend (vérification du claim role dans le JWT).

Rôle	Valeur en base	Droits backend	Pages Angular
Utilisateur	UTILISATEUR	Lire, réserver tickets, annuler ses tickets	/, /tickets, /artistes, /evenement/:id
Organisateur	ORGANISATEUR	CRUD ses événements, gérer artistes, voir stats	/, /organisateur, /artistes
Administrateur	ADMIN	Accès total — tous endpoints	/, /admin, /organisateur, /artistes

AuthGuard Angular

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  canActivate(route: ActivatedRouteSnapshot): boolean {
    const token = localStorage.getItem('token');
    const user = JSON.parse(localStorage.getItem('user') || '{}');
    const requiredRoles = route.data['roles'] as string[];
    if (!token) { this.router.navigate(['/login']); return false; }
    if (requiredRoles && !requiredRoles.includes(user.role)) {
      this.router.navigate(['/']); return false;
    }
    return true;
  }
}
```


Configuration des routes protégées (app.routes.ts)

```
{ path: 'organisateur', component: OrganisateurComponent,
  canActivate: [AuthGuard],
  data: { roles: ['ORGANISATEUR', 'ADMIN'] } },
{ path: 'admin', component: AdminComponent,
  canActivate: [AuthGuard],
  data: { roles: ['ADMIN'] } },
```

3.6 Upload et affichage des images

Les images des événements et des artistes sont gérées via des URLs (stockage externe ou serveur de fichiers). L'interface Angular propose un champ URL avec prévisualisation en temps réel lors de la saisie.

Champ imageUrl dans les formulaires Angular

```
<!-- Dans le formulaire de création d'événement -->
<input [(ngModel)]="event.imageUrl" placeholder="URL de l'image" />
<img *ngIf="event.imageUrl" [src]="event.imageUrl"
    (error)="event.imageUrl = ''"
    class="preview-img" />
```

Affichage sécurisé — fallback image

```
<!-- Composant EventCard -->
<img [src]="event.imageUrl || 'assets/default-event.jpg'"
    (error)="onImageError($event)"
    [alt]="event.nom" />
```

Pour un upload multipart réel, l'endpoint backend utiliserait `@Consumes("multipart/form-data")` avec la bibliothèque Apache Commons FileUpload, et stockerait les fichiers dans un répertoire dédié (ex. `/uploads/`) servi en statique par le container Grizzly.

3.7 Architecture Angular 18 — Structure des composants

Le frontend utilise Angular 18 en mode standalone (sans NgModule), avec lazy loading sur toutes les routes, des Signals pour la gestion d'état réactif, et un `HttpInterceptor` fonctionnel pour l'injection automatique du JWT.

Structure du projet Angular

Dossier / Fichier	Rôle
app.config.ts	Configuration globale : provideRouter, provideHttpClient + authInterceptor
app.routes.ts	Routes avec lazy loading + guards (authGuard, adminGuard, organisateurGuard, guestGuard)
models/index.ts	Interfaces TypeScript : Evenement, Artiste, Ticket, Utilisateur, Organisateur, AuthResponse...
services/api.service.ts	Service HTTP générique : get/post/put/delete — BASE_URL = '/api' (proxy vers :8081)
services/auth.service.ts	Authentification : login, register, logout, signals isLoggedIn/isAdmin/isOrganisateur
services/evenement.service.ts	CRUD événements + Signal state + filteredEvenements computed + mock data
services/ticket.service.ts	Achat, annulation, PDF jsPDF, QR Code qrcode, validTickets/usedTickets computed
services/artiste.service.ts	CRUD artistes + Signal state + mock data
core/guards/auth.guard.ts	authGuard, adminGuard, organisateurGuard, guestGuard (CanActivateFn)
pages/auth/auth.interceptor.ts	HttpInterceptorFn : injection header Authorization: Bearer <token>
pages/home/	Page d'accueil : Hero + StatsBar + CategoryFilter + EventList + Toast
pages/login/ & register/	Formulaires d'authentification (FormsModule + ngModel)
pages/tickets/	Mes Tickets : billets valides + historique + annulation + téléchargement PDF
pages/ticket-control/	Interface de contrôle : saisie ID → vérification → validation (statut UTILISE)
pages/organisateur/	Espace organisateur : tableau événements + formulaire création/édition + suppression
pages/admin/	Panneau Admin : dashboard KPIs + onglets événements/artistes/utilisateurs
pages/artiste-detail/	Galerie artistes publique avec liste des prochains événements par artiste
pages/artistes-admin/	CRUD artistes pour l'administrateur
components/reservation-modal/	Modal réservation 3 étapes : detail → confirm → loading → success
components/navbar/	Barre de navigation responsive (menuOpen toggle, liens conditionnels par rôle)

Gestion d'état avec Angular Signals

Tous les services utilisent les Signals Angular 18 pour l'état réactif. Les computed signals dérivent automatiquement l'état filtré sans besoin de subscription manuelle.

```
// EvenementService - Signal state + computed filtrage
private _evenements = signal<Evenement[]>([]);
private _selectedCategory = signal<string>('all');
private _searchQuery = signal<string>('');

readonly filteredEvenements = computed(() => {
  const cat = this._selectedCategory().toLowerCase();
  const q = this._searchQuery().toLowerCase();
  return this._evenements().filter(e => {
    const matchCat = cat === 'all' || e.genre?.toLowerCase() === cat
                        || e.categorie?.toLowerCase() === cat;
    const matchQ = !q || e.nom.toLowerCase().includes(q)
                    || e.lieu?.toLowerCase().includes(q);
    return matchCat && matchQ;
  });
});
```

Proxy Angular → Backend (proxy.conf.json)

Le frontend utilise un proxy Angular en développement. Toutes les requêtes vers /api sont transparentement redirigées vers le backend JAX-RS sur le port 8081. Cela évite les erreurs CORS en développement.

```
// proxy.conf.json
{ "/api": {
  "target": "http://localhost:8081",
  "changeOrigin": true,
  "pathRewrite": { "^/api": "" }
} }
```

```
// Ainsi : api.get('evenements') → GET /api/evenements → proxy → GET
http://localhost:8081/evenements
```

HttpInterceptor fonctionnel — authInterceptor

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('token');
  if (token) {
    return next(req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    }));
  }
  return next(req);
};
```

```
// Enregistrement dans app.config.ts
provideHttpClient(withInterceptors([authInterceptor]))
```

Guards de navigation — résumé

Guard	Condition	Redirection si refus	Routes protégées
authGuard	isLoggedIn() = true	/login	/tickets
adminGuard	isAdmin() = true (rôle ADMIN)	/	/admin, /admin/artistes/new
organisateurGuard	isOrganisateur() OR isAdmin()	/	/organisateur
guestGuard	!isLoggedIn() (non connecté)	/	/login, /register

Dépendances npm clés (package.json)

Package	Version	Usage
@angular/core	^18.0.0	Framework — standalone components, signals, computed
@angular/router	^18.0.0	Routing — lazy loading, CanActivateFn guards
@angular/forms	^18.0.0	FormsModule — ngModel pour tous les formulaires
rxjs	~7.8.0	Observables, tap, forkJoin (achats multiples tickets)
jspdf	^4.2.1	Génération PDF tickets — format A5 landscape, design coloré
qrcode	^1.5.4	Génération QR Code PNG base64 — encodage ID ticket
@types/qrcode	^1.5.6	Types TypeScript pour l'API qrcode
zone.js	~0.14.3	Détection de changements Angular
typescript	~5.4.2	Compilateur TypeScript (strict mode)

Mode hors-ligne — Fallback mock data

Si le backend est indisponible, chaque service dispose d'une méthode loadMock() injectant des données de démonstration. Les composants l'appellent dans le bloc error: de leurs subscriptions.

```
// Exemple dans TicketsComponent
this.tk.loadMyTickets().subscribe({
  next: () => this.loading.set(false),
  error: () => { this.tk.loadMock(); this.loading.set(false); }
});
```

Les mock data couvrent 6 événements (tous genres), 6 artistes et des tickets de test. Le basculement est transparent pour l'utilisateur — aucune erreur n'est affichée.

4. DÉMARRAGE DE L'APPLICATION

4.1 Prérequis

Outil	Version recommandée	Usage
Docker Desktop	v28+	Hébergement du backend JAX-RS et de la base de données
Node.js	v18 ou v20 LTS	Environnement d'exécution JavaScript pour Angular
npm	v9+	Gestionnaire de paquets Angular
Angular CLI	v18	Serveur de développement et build Angular
Java JDK	v17+	Compilation du backend (si reconstruction manuelle)
Maven	v3.8+	Gestion des dépendances Java
Navigateur web	Chrome / Edge / Firefox	Accès à l'interface utilisateur

4.2 Lancement du Backend (Docker)

Le backend est entièrement conteneurisé. Deux containers sont démarrés par docker-compose : le serveur JAX-RS (port 8081) et la base de données (port 9001 pour HSQLDB, ou 3306 pour MySQL).

Étape	Commande / Action	Résultat attendu
1 — Accéder au dossier backend	cd chemin/vers/JaxRSOpenAPI	Terminal positionné dans le projet
2 — Lancer Docker Compose	docker-compose up	Démarrage des 2 containers
3 — Vérifier le démarrage	Chercher dans les logs :	INFO: JAX-RS based micro-service running!
4 — Tester l'API	Ouvrir http://localhost:8081/evenements	Réponse JSON ([]) ou liste d'événements)
5 — Swagger UI	Ouvrir http://localhost:8081/swagger-ui	Documentation interactive de l'API

Structure docker-compose.yml

```
services:
  backend:
    build: .
    ports: ["8081:8081"]
    depends_on: [db]
    environment:
      DB_HOST: db
      DB_PORT: 3306
  db:
    image: mysql:8
    ports: ["3306:3306"]
```

```
environment:
  MYSQL_DATABASE: eventhub
  MYSQL_ROOT_PASSWORD: root
```

4.3 Lancement du Frontend (Angular)

Étape	Commande	Résultat
1 — Accéder au dossier Angular	cd chemin/vers/eventhub	Terminal dans le projet Angular
2 — Installer les dépendances	npm install	node_modules/ créé
3 — Lancer le serveur de dev	ng serve	Compilation et démarrage
4 — Ouvrir l'application	http://localhost:4200	Interface EventHub affichée

⚠ Ordre de démarrage obligatoire : 1) Docker Desktop → 2) docker-compose up → 3) ng serve → 4) Ouvrir http://localhost:4200 Si le backend est hors ligne, Angular bascule automatiquement sur les données de démonstration intégrées (mock data) — aucune erreur n'est visible pour l'utilisateur.

5. COMPTES UTILISATEURS DISPONIBLES

Trois comptes préconfigurés permettent de tester toutes les fonctionnalités sans création préalable. Lors de la connexion, le nom d'utilisateur suffit (le mot de passe peut être quelconque en mode démo).

Utilisateur	Email / Login	Rôle	Accès
admin	admin	ADMIN	Accès total — Dashboard, gestion globale, tous endpoints
sophie	sophie	UTILISATEUR	Réservation, gestion de ses tickets, QR code, PDF
jean	jean	ORGANISATEUR	Création et gestion d'événements, statistiques, artistes

6. FONCTIONNALITÉS — GUIDE UTILISATEUR

6.1 Page d'accueil (/)

Page principale de découverte des événements. Elle est accessible sans connexion et constitue le point d'entrée de l'application.

- Affichage des événements sous forme de cartes (image, titre, date, lieu, prix)
- Barre de recherche temps réel par nom d'événement ou lieu
- Filtres par catégorie musicale : Électronique, Jazz, Rock, House, Classique, Hip-Hop
- Clic sur une carte → page de détail /evenement/:id
- Bouton Réserver rapide sur chaque carte (redirige vers /login si non connecté)
- Mode hors-ligne : basculement automatique vers les données de démonstration

6.2 Connexion et Inscription

Page de connexion (/login)

8. Saisir le nom d'utilisateur (email) dans le champ dédié
9. Saisir le mot de passe
10. Cliquer sur Se connecter
11. Redirection vers la page d'accueil avec le rôle actif

Page d'inscription (/register)

12. Remplir les champs : prénom, nom, nom d'utilisateur, mot de passe, confirmation
13. Choisir le type de compte : Utilisateur ou Organisateur
14. Cliquer sur Créer mon compte
15. Redirection automatique vers la page d'accueil

Flux technique inscription : POST /auth/register → validation des données → hachage du mot de passe (BCrypt) → création en base → génération JWT → retour LoginResponse { token, user }

6.3 Réservation de tickets (processus en 3 étapes)

Le processus de réservation est guidé et se déroule en trois étapes successives depuis la page de détail d'un événement.

Étape	Page / Composant	Actions utilisateur	Données échangées
1 — Sélection	Page détail /evenement/:id	Choisir la quantité (1–10). Le total est calculé automatiquement.	Lecture GET /evenements/{id} — affichage jauge disponibilité
2 — Confirmation	Modal de confirmation	Vérifier le récapitulatif : événement, quantité, total à payer.	POST /tickets avec { eventId, utilisateurId, quantite }
3 — Succès	Confirmation visuelle	Un message de succès s'affiche. Les tickets	Mise à jour nbTicketsVendus, génération des numéros de place

		apparaissent dans Mes Tickets.	
--	--	--------------------------------	--

Payload POST /tickets

```
{
  "evenementId": 3,
  "utilisateurId": 12,
  "quantite": 2,
  "prixUnitaire": 35.00
}
```

Logique côté TicketService (backend)

```
public List<Ticket> acheterTickets(Long evenementId, Long userId, int quantite) {
    Evenement e = evenementDao.findById(evenementId);
    if (e.getNbTicketsVendus() + quantite > e.getCapacite())
        throw new IllegalStateException("Plus de places disponibles");
    List<Ticket> tickets = new ArrayList<>();
    for (int i = 0; i < quantite; i++) {
        Ticket t = new Ticket(generateNumeroPlace(), e.getPrix(), e);
        t.setStatut(StatutTicketEnum.VALIDE);
        t.setDateAchat(LocalDate.now());
        tickets.add(ticketDao.save(t));
    }
    e.setNbTicketsVendus(e.getNbTicketsVendus() + quantite);
    return tickets;
}
```

6.4 QR Code et génération PDF

QR Code par ticket

Chaque ticket valide dispose d'un QR code unique généré côté frontend (Angular) via la bibliothèque qrcode.

- Le contenu du QR code encode : ID ticket + ID événement + ID utilisateur + statut
- Format de la donnée encodée : EVENTHUB-TICKET-{idTicket}-{evenementId}-{userId}
- Le QR code s'affiche directement sur la carte ticket dans Mes Tickets
- Il est intégré dans le PDF téléchargeable (encodé en base64 PNG)

Implémentation Angular du QR Code

```
import QRCode from 'qrcode';

async generateQrCode(ticket: Ticket): Promise<string> {
    const data = `EVENTHUB-TICKET-${ticket.idTicket}-${ticket.evenement.idEvenement}-${ticket.utilisateur.idPersonne}`;
    return await QRCode.toDataURL(data, {
        width: 200, margin: 1,
        color: { dark: '#2D211C', light: '#FFFFFF' }
    });
}
```


Génération PDF

Le téléchargement du billet au format PDF est généré entièrement côté Angular via la bibliothèque jsPDF + html2canvas. Le PDF intègre toutes les informations du ticket ainsi que le QR code.

- Bibliothèques utilisées : jsPDF + html2canvas
- Contenu du PDF : logo EventHub, nom de l'événement, date et lieu, nom du spectateur, numéro de billet, QR code haute résolution, statut du ticket
- Format de sortie : A4 portrait, nommé ticket-{idTicket}.pdf

Code de génération PDF (Angular)

```
async downloadPdf(ticket: Ticket) {  
  const element = document.getElementById('ticket-' + ticket.idTicket);  
  const canvas = await html2canvas(element, { scale: 2 });  
  const imgData = canvas.toDataURL('image/png');  
  const pdf = new jsPDF({ orientation: 'portrait', unit: 'mm', format: 'a4' });  
  pdf.addImage(imgData, 'PNG', 10, 10, 190, 0);  
  pdf.save(`ticket-${ticket.idTicket}.pdf`);  
}
```

6.5 Contrôle des tickets

La fonctionnalité de contrôle permet à un organisateur ou administrateur de valider un ticket à l'entrée d'un événement, en changeant son statut de VALIDE à UTILISE.

Statut	Valeur enum	Description	Actions possibles
☒ Valide	VALIDE	Ticket acheté, non encore utilisé	Annuler, Télécharger PDF, Scanner QR
☒ Utilisé	UTILISE	Ticket scanné/validé à l'entrée	Consulter uniquement (PDF désactivé)
☒ Annulé	ANNULE	Annulé par l'utilisateur	Consulter dans l'historique
☒ Remboursé	REMBOURSE	Annulation avec remboursement	Consulter dans l'historique

Endpoint de validation d'un ticket

```
PUT /tickets/{id}  
{  
  "statut": "UTILISE",  
  "dateAnnulation": null  
}
```

Annulation par l'utilisateur

16. Aller dans Mes Tickets via la barre de navigation
17. Cliquer sur le bouton ✕ Annuler sur le ticket souhaité
18. Confirmer dans la fenêtre de dialogue
19. Le ticket passe au statut ANNULE et est déplacé dans la section Historique

Règle métier : un ticket UTILISE ou ANNULE ne peut plus être annulé. La vérification est faite côté backend avant la mise à jour (lève `IllegalStateException` si la transition est invalide).

6.6 Gestion des artistes

Page Artistes publique (/artistes)

Accessible sans connexion depuis la barre de navigation. Présente la galerie complète des artistes référencés.

- Photo / visuel de l'artiste (imageUrl)
- Nom complet et style artistique
- Nationalité et score de popularité
- Liste des prochains événements avec date — clic → page de détail de l'événement

CRUD Artistes (Organisateur / Admin)

Opération	Endpoint	Accès	Payload / Paramètres
Créer un artiste	POST /artistes	ORGANISATEUR, ADMIN	{ nom, prenom, styleArtistique, nationalite, description, popularite, siteWeb, imageUrl }
Modifier un artiste	PUT /artistes/{id}	ORGANISATEUR, ADMIN	Mêmes champs (partiels acceptés)
Supprimer un artiste	DELETE /artistes/{id}	ADMIN uniquement	Aucun body requis
Voir tous les artistes	GET /artistes	Public	Réponse : List<Artiste>
Artiste par ID	GET /artistes/{id}	Public	Réponse : Artiste + événements associés
Filtrer par style	GET /artistes?style=Jazz	Public	Query param styleArtistique

Associer un artiste à un événement (Formulaire Organisateur)

20. Dans l'Espace Organisateur, créer ou modifier un événement
21. Dans la section Artistes, rechercher par nom dans le champ dédié
22. Cliquer sur + Ajouter pour créer l'association (POST /evenements/{id}/artistes/{artistId})
23. L'artiste apparaît dans la liste des artistes de l'événement
24. Cliquer sur × pour retirer l'association (DELETE /evenements/{id}/artistes/{artistId})

6.7 Espace Organisateur (/organisateur)

Accessible aux rôles ORGANISATEUR et ADMIN. Interface de gestion complète des événements organisée en deux onglets.

Onglet	Contenu	Actions disponibles
 Mes événements	Tableau de tous les événements de l'organisateur avec statistiques en temps réel	Modifier  , Supprimer  , Voir  , Barre de progression des places
 Créer / Modifier	Formulaire complet de création ou d'édition d'événement	Nom, date, heure, lieu, catégorie, prix, capacité, description, URL image, gestion des artistes

Formulaire de création d'événement — Champs

Champ	Type	Validation	Obligatoire
Nom de l'événement	text	Min. 3 caractères	Oui
Date	date (LocalDate)	Date future uniquement	Oui
Heure	time	Format HH:mm	Oui
Lieu / Salle	text	Min. 3 caractères	Oui
Catégorie musicale	select	Parmi la liste définie	Oui
Prix unitaire (€)	number	> 0	Oui
Capacité (places)	number	> 0, entier	Oui
Description	textarea	Max. 2000 caractères	Non
URL de l'image	url	URL valide, prévisualisation temps réel	Non
Artistes associés	recherche dynamique	Sélection depuis la liste existante	Oui

6.8 Panneau Administrateur (/admin) + Statistiques

Réservé au rôle ADMIN. Donne une vision globale et en temps réel de toute la plateforme, organisée en quatre onglets.

Onglet 1 — Dashboard KPIs

KPI	Calcul	Source
Total événements	COUNT(eventements)	GET /eventements
Tickets vendus	SUM(nbTicketsVendus)	GET /eventements — agrégation frontend
Artistes référencés	COUNT(artistes)	GET /artistes
Revenus estimés (€)	SUM(nbTicketsVendus × prix)	Calcul frontend sur données événements
Taux de remplissage (%)	nbTicketsVendus / capacite × 100	Par événement — barre de progression

Onglets 2, 3, 4 — Gestion globale

- Onglet Événements : liste complète avec possibilité de supprimer ou d'accéder au détail
- Onglet Artistes : galerie complète, création/modification/suppression
- Onglet Utilisateurs : liste de tous les comptes avec leur rôle respectif

Statistiques Organisateur (dans l'espace organisateur)

- Tickets vendus par événement
- Revenus générés ($\text{nbTicketsVendus} \times \text{prixUnitaire}$) par événement
- Taux de remplissage en temps réel (barre de progression colorée)
- Comparaison entre événements actifs

7. API BACKEND — ENDPOINTS COMPLETS

Base URL : `http://localhost:8081` — **Swagger UI :**
`http://localhost:8081/swagger-ui`

7.1 Auth — Authentification

Méthode	URL	Description	Auth requise
POST	/auth/login	Connexion — retourne JWT + infos utilisateur	Non
POST	/auth/register	Inscription — crée compte + retourne JWT	Non

Exemple POST /auth/login

```
Body : { "email": "sophie", "password": "motdepasse" }  
Réponse 200 : { "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ", "user": { "idPersonne": 2, "nom": "Martin", "role": "UTILISATEUR" } }
```

7.2 Événements (/evenements)

Méthode	URL	Description	Auth
GET	/evenements	Tous les événements	Non
GET	/evenements/{id}	Événement par ID	Non
GET	/evenements/search?nom=&genre=&lieu=	Recherche multicritères	Non
GET	/evenements/by-date/{date}	Événements à une date donnée (YYYY-MM-DD)	Non
GET	/evenements/populaires/{seuil}	Événements dont popularite >= seuil	Non
GET	/evenements/with-artistes	Événements avec leurs artistes inclus	Non
POST	/evenements	Créer un événement	ORGANISATEUR, ADMIN
PUT	/evenements/{id}	Modifier un événement	ORGANISATEUR, ADMIN
DELETE	/evenements/{id}	Supprimer un événement	ORGANISATEUR, ADMIN
POST	/evenements/{id}/artistes/{artisteld}	Associer un artiste	ORGANISATEUR, ADMIN
DELETE	/evenements/{id}/artistes/{artisteld}	Retirer un artiste	ORGANISATEUR, ADMIN

7.3 Artistes (/artistes)

Méthode	URL	Description	Auth
GET	/artistes	Tous les artistes	Non
GET	/artistes/{id}	Artiste par ID	Non
GET	/artistes?style={style}	Filtrer par style artistique	Non
GET	/artistes?nationalite={nat}	Filtrer par nationalité	Non
POST	/artistes	Créer un artiste	ORGANISATEUR, ADMIN
PUT	/artistes/{id}	Modifier un artiste	ORGANISATEUR, ADMIN
DELETE	/artistes/{id}	Supprimer un artiste	ADMIN

7.4 Tickets (/tickets)

Méthode	URL	Description	Auth
GET	/tickets	Tous les tickets (admin)	ADMIN
GET	/tickets/utilisateur/{id}	Tickets d'un utilisateur	Utilisateur connecté
GET	/tickets/{id}	Ticket par ID	Utilisateur connecté
POST	/tickets	Acheter un ou plusieurs tickets	Utilisateur connecté
PUT	/tickets/{id}	Modifier le statut (UTILISE, ANNULE...)	Utilisateur connecté
DELETE	/tickets/{id}	Supprimer un ticket (admin)	ADMIN

7.5 Utilisateurs & Organismes

Méthode	URL	Description	Auth
GET	/utilisateurs	Tous les utilisateurs	ADMIN
GET	/utilisateurs/{id}	Utilisateur par ID	ADMIN ou self
POST	/utilisateurs	Créer un utilisateur (bypass register)	ADMIN
PUT	/utilisateurs/{id}	Modifier un utilisateur	ADMIN ou self
DELETE	/utilisateurs/{id}	Supprimer un compte	ADMIN
GET	/organismes	Tous les organismes	ADMIN
POST	/organismes	Créer un organisme	ADMIN
PUT	/organismes/{id}	Modifier un organisme	ADMIN ou self

8. CRUD COMPLET — DÉTAIL DES OPÉRATIONS

Cette section détaille le comportement de chaque opération CRUD, de la validation des données côté Angular jusqu'à la persistance JPA côté backend.

Create — Création

Étape	Couche	Détail
1. Saisie formulaire	Angular (Component)	Validation réactive (Validators.required, min, pattern)
2. Soumission HTTP	Angular (Service)	POST JSON via HttpClient avec header Authorization: Bearer
3. Désérialisation	JAX-RS (Resource)	Jackson désérialise le JSON en entité Java
4. Validation métier	Service Java	Vérifications business (capacité, dates futures, unicité)
5. Persistance	DAO (JPA)	entityManager.persist(entity) + transaction commit
6. Retour 201	JAX-RS (Resource)	Response.status(201).entity(created).build()
7. Mise à jour UI	Angular (Component)	Ajout à la liste locale, navigation ou toast de succès

Read — Lecture

- Chargement initial : `ngOnInit()` → `service.getAll()` → GET /evenements
- Lecture par ID : `ActivatedRoute.params` → `service.getById(id)` → GET /evenements/{id}
- Recherche temps réel : `FormControl.valueChanges` → `debounceTime(300)` → GET /evenements/search?nom=...
- Filtre par catégorie : traitement local sur la liste déjà chargée (filtre JavaScript côté Angular)

Update — Modification

```
// Angular — pré-remplissage du formulaire avec les données existantes
this.eventForm.patchValue({
  nom: event.nom, date: event.date, lieu: event.lieu,
  prix: event.prix, capacite: event.capacite,
  description: event.description, imageUrl: event.imageUrl
});
```

```
// Backend — mise à jour partielle dans EvenementService
public Evenement updateEvenement(Long id, Evenement data) {
    Evenement existing = dao.findById(id);
    if (existing == null) throw new RuntimeException("Non trouvé");
    if (data.getNom() != null) existing.setNom(data.getNom());
    if (data.getPrix() != null) existing.setPrix(data.getPrix());
    if (data.getDescription() != null)
        existing.setDescription(data.getDescription());
    // ... autres champs
    return dao.update(existing);
}
```

Delete — Suppression

- Côté Angular : dialogue de confirmation obligatoire avant appel DELETE
- Côté backend : vérification que l'entité existe (404 si non trouvée)
- Cascade : la suppression d'un événement supprime ses tickets associés (CascadeType.PERSIST — à étendre à CascadeType.REMOVE si besoin)
- Côté Angular : retrait de la liste locale après confirmation de la réponse 204

9. STATISTIQUES ET KPIs

Les statistiques sont calculées à deux niveaux : au niveau global (dashboard Admin) et au niveau individuel (espace Organisateur).

Dashboard Admin — KPIs globaux

Indicateur	Formule	Mise à jour
Nombre d'événements	<code>evenements.length</code>	À chaque chargement de la page
Total tickets vendus	$\Sigma \text{ evenement.nbTicketsVendus}$	Temps réel via API
Total artistes	<code>artistes.length</code>	À chaque chargement
Revenus estimés (€)	$\Sigma (\text{nbTicketsVendus} \times \text{prix})$	Calculé frontend sur les données API
Taux de remplissage moyen	$\Sigma(\text{nbTicketsVendus}/\text{capacite}) / \text{nbEvenements} \times 100$	Calculé frontend
Événements complets (>90%)	Filter evenements where taux > 90%	Calculé frontend

Statistiques par événement (Organisateur)

Métrique	Affichage	Source
Places vendues	<code>nbTicketsVendus / capacite</code>	GET <code>/evenements/{id}</code>
Places restantes	<code>capacite - nbTicketsVendus</code>	Calculé frontend
Taux de remplissage	Barre de progression en %	Calculé frontend
Revenu généré	<code>nbTicketsVendus × prix (€)</code>	Calculé frontend
Statut de l'événement	COMPLET / DISPONIBLE / BIENTÔT COMPLET	Seuils : 100% / <75% / ≥75%

Évolution implémentée — nbTicketsVendus

Le champ `nbTicketsVendus` de l'entité `Evenement` est incrémenté côté backend à chaque achat de ticket. Il est décrémenté lors d'une annulation si le ticket était VALIDE.

```
// Lors de l'achat (TicketService.acheterTickets)
evenement.setNbTicketsVendus(evenement.getNbTicketsVendus() + quantite);
evenementDao.update(evenement);
```

```
// Lors de l'annulation (TicketService.annulerTicket)
if (ticket.getStatut() == StatutTicketEnum.VALIDE) {
    evenement.setNbTicketsVendus(Math.max(0, evenement.getNbTicketsVendus() - 1));
    evenementDao.update(evenement);
}
```

```
}  
ticket.setStatut(StatutTicketEnum.ANNULE);  
ticket.setDateAnnulation(LocalDateTime.now());
```

10. PROBLÈMES RENCONTRÉS ET SOLUTIONS

Cette section documente les principaux obstacles techniques rencontrés lors du développement d'EventHub, ainsi que les solutions adoptées.

P1 — Récursion infinie JSON avec relations bidirectionnelles JPA

	Détail
Problème	La sérialisation Jackson d'un Evenement incluait ses Artistes, qui incluait à leur tour leurs Evenements → StackOverflowError
Cause	Relation @ManyToMany bidirectionnelle sans gestion de la récursion
Solution	Ajouter @JsonIgnore sur le champ List<Evenement> côté Artiste + @JsonIgnoreProperties({"tickets", "artistes"}) sur les relations inversées
Code	@JsonIgnore @ManyToMany(mappedBy = "artistes") private List<Evenement> evenements;

P2 — Erreurs CORS bloquant les appels Angular → JAX-RS

	Détail
Problème	Le navigateur bloquait les requêtes cross-origin vers http://localhost:8081 (erreur CORS dans la console)
Cause	Le serveur JAX-RS ne retournait pas les headers CORS nécessaires
Solution	Implémentation d'un CorsFilter et d'un CorsOptionsRequestFilter interceptant toutes les requêtes OPTIONS
Headers ajoutés	Access-Control-Allow-Origin: * Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS Access-Control-Allow-Headers: Authorization, Content-Type

P3 — Sérialisation des types Java 8 (LocalDate, LocalDateTime)

	Détail
Problème	Jackson sérialisait LocalDate en tableau [2025, 6, 15] au lieu de "2025-06-15"
Cause	Absence du module JavaTimeModule dans la configuration Jackson
Solution	JacksonConfig.java → ObjectMapper avec registerModule(new JavaTimeModule()) et WRITE_DATES_AS_TIMESTAMPS à false
Code	mapper.registerModule(new JavaTimeModule()); mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);

P4 — Ports déjà utilisés au démarrage

Port	Commande diagnostic (Windows)	Résolution
8081 (backend)	netstat -ano findstr :8081	taskkill /PID <PID> /F — ou docker-compose down puis up
4200 (Angular)	netstat -ano findstr :4200	taskkill /PID <PID> /F — ou ng serve --port 4201
3306 (MySQL)	netstat -ano findstr :3306	Arrêter MySQL local si installé, ou changer le port dans docker-compose

P5 — Token JWT expiré sans feedback utilisateur

	Détail
Problème	Après 24h, le token expire et les requêtes retournent 401 sans que l'utilisateur soit informé ni redirigé
Solution	HttpInterceptor Angular interceptant les réponses 401 → suppression du token + redirection vers /login + toast d'information
Code	<pre>if (err.status === 401) { localStorage.removeItem('token'); this.router.navigate(['/login']); }</pre>

P6 — Données non rechargées après création/modification

	Détail
Problème	Après création ou modification d'un événement, la liste dans l'espace Organisateur ne se rafraîchissait pas automatiquement
Solution	Appel de this.loadEvenements() systématiquement dans le .subscribe() de chaque opération POST/PUT/DELETE
Pattern adopté	<pre>create().subscribe({ next: () => { this.showSuccess(); this.loadEvenements(); }, error: (e) => this.showError(e) })</pre>

P7 — Gestion du fallback image manquante

	Détail
Problème	Si une URL d'image est invalide ou inaccessible, une icône cassée s'affiche sur les cartes événements
Solution	Gestionnaire (error) sur chaque balise → remplacement par l'image par défaut assets/default-event.jpg
Code HTML	<code></code>

Tableau de bord des problèmes

#	Problème	Sévérité	Statut
P1	Récursion infinie JSON (@JsonIgnore)	Critique	☑ Résolu
P2	Erreurs CORS Angular → JAX-RS	Critique	☑ Résolu
P3	Sérialisation LocalDate Jackson	Majeur	☑ Résolu
P4	Ports 8081 / 4200 déjà utilisés	Majeur	☑ Résolu (doc + scripts)
P5	Token JWT expiré sans redirection	Moyen	☑ Résolu (interceptor)
P6	Liste non rafraîchie après CRUD	Moyen	☑ Résolu
P7	Image manquante affichée cassée	Mineur	☑ Résolu (fallback)

11. CONCLUSION

EventHub est une application web complète et fonctionnelle de gestion et réservation d'événements musicaux. Développée en équipe dans le cadre du cours SIR, elle met en œuvre l'ensemble des composantes d'une architecture moderne trois-tiers.

Bilan technique

Composante	Technologie	Implémentation
Frontend	Angular 18 + TypeScript	Composants, services, guards, interceptors, lazy loading
Backend REST	JAX-RS / Jakarta EE	Resources, services, DAO pattern, filters
Persistence	JPA / Hibernate	Entités, relations (ManyToMany,ManyToOne,OneToMany), NamedQueries
Base de données	MySQL / HSQLDB	Schéma automatique via JPA, données de test via insertions
Authentification	JWT (JJWT)	Génération HS256, filtre de validation, HttpInterceptor Angular
Autorisation	RBAC (3 rôles)	AuthGuard Angular + vérification role dans JWT côté backend
QR Code	qrcode (npm)	Génération PNG base64, intégration PDF
Génération PDF	jsPDF + html2canvas	Rendu HTML → canvas → PDF A4 téléchargeable
Upload images	URLs externes + preview	Prévisualisation temps réel, fallback sur erreur
Conteneurisation	Docker + docker-compose	Backend + BDD en containers, démarrage en une commande
Documentation API	Swagger / OpenAPI 3	Annotations @Operation, @Tag, UI interactive

Fonctionnalités livrées

- CRUD complet : événements, artistes, tickets, utilisateurs, organisateurs
- Relation ManyToMany Événement ↔ Artiste avec table de jointure
- Authentification JWT avec gestion des rôles (Utilisateur / Organisateur / Admin)
- Réservation de tickets en 3 étapes avec vérification de capacité
- Génération de QR code unique par ticket
- Export PDF téléchargeable pour chaque ticket
- Contrôle des tickets (changement de statut : VALIDE → UTILISE)
- Upload et affichage des images avec fallback
- Statistiques et KPIs en temps réel (dashboard Admin + espace Organisateur)
- Panneau d'administration complet
- Mode hors-ligne avec données de démonstration